TECNOLÓGICO NACIONAL DE MEXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
ENERO - JUNIO 2026

NOMBRE DE LA CARRERA:
INGENIERÍA EN SISTEMAS COMPUTACIONALES

NOMBRE DE LA MATERIA:
PATRONES DE DISEÑO DE SOFTWARE

TÍTULO DEL TRABAJO:
PATRÓN DE DISEÑO DE COMPORTAMIENTO: ESTADO

UNIDAD A EVALUAR:
UNIDAD 4

NOMBRE DEL ALUMNO:
RODRIGUEZ MORALES SUSANA ABIGAIL - 22210346

NOMBRE DEL MAESTRO:
MARIBEL GUERRERO LUIS

Índice

Introducción 4
Diagrama de estados 5
Interfaz 6
- Máquina expendedora 6
  - Código 7
    - Agregar context 7
    - Referencias 7
    - Constructor 7
    - Botón aceptar 8
    - Botón cancelar 9
    - Temporizador para dispensar 10
    - Validación de etiquetas 11
- Monedas 12
  - Código 12
- Billetes 13
  - Código 13

Implementación del patrón State 14
- State 14
- Context 15
  - Constructor 15
  - Métodos que pertenecen a context 16
  - Reiniciar 16
- Estado Espera 17
  - Ingresar dinero 17
  - Consultar precio 18
  - Confirmar 18
- Estado Recibir dinero 19
  - Ingresar dinero 19
  - Confirmar 19
  - Cancelar 20
- Estado Producto Listo 21
  - Elegir producto 21
  - Confirmar 21
  - Cancelar 21
- Estado Dispensando 22
  - Confirmar 22

Pruebas 23
Caso 1 23
Caso 2 23
Caso 3 23
Caso 4 24
Caso 5 24
Caso 6 25
Conclusión 26

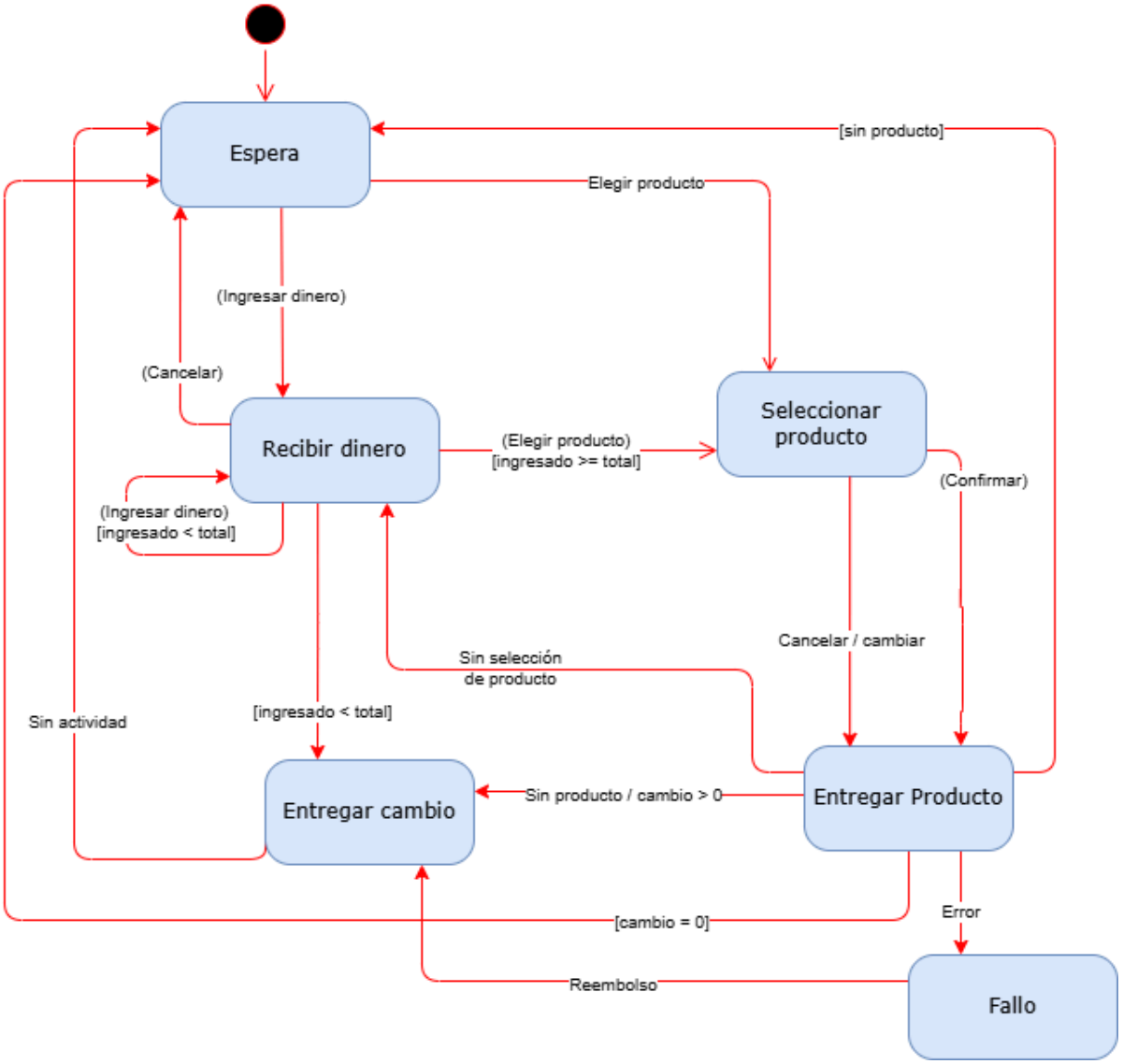
Introducción

Patrón State

También conocido como objeto de estado o máquina de estados es utilizado cuando un objeto puede cambiar su comportamiento según su estado interno. Es útil cuando un objeto puede tener múltiples estados que deben ser gestionados para que tenga transiciones flexibles.

En este proyecto la implementación del patrón state permitirá simular el comportamiento de una máquina expendedora, representando los estados en los que puede estar y acciones que se pueden realizar, según el usuario interactúe.

Diagrama de estados



Interfaz

Máquina expendedora

Interfaz con la que el usuario interactuara para realizar una compra.
De un lado están los productos que se podrán comprar y la compuerta donde se toma el producto, y al otro lado está la pantalla, el teclado, dos ranuras: la primera para ingresar monedas y la segunda para ingresar billetes, como último está la charola del cambio, todos estos elementos están para la interacción del usuario con la máquina.

Código

Agregar context

Le da el estado en el que iniciará la máquina.

```
8 referencias
public partial class Form1 : Form
{
    //agrega el context a la forma y lo inicia en estado de espera
    public Context contexto = new Context(new StateEspera());
```

Referencias

Referencia a formulario de monedas y billetes.

```
    //forms monedas y billetes
    Form2 monedas;
    Form3 billetes;
```

Constructor

Inicializa el formulario, los temporizadores, la charola del cambio y las etiquetas que muestran el stock.

```
1 referencia
public Form1()
{
    InitializeComponent();

    //Timer para mostrar precio
    tmPrecio.Interval = 500;
    tmPrecio.Tick += TmPrecio_t;

    //Timer para limpiar pantalla
    tmLimpiar.Interval = 5000;
    tmLimpiar.Tick += TmLimpia_t;


    //dispensar producto
    tmDispensar.Interval = 3000;
    tmDispensar.Tick += TmDispensar_Tick;

    //la charola del cambio inicia no visible
    p2Cambio.Visible = false;

    LimpiarPanel();

    MostrarStock();
}
```

Botón aceptar

Confirma la acción según el estado en el que se encuentre. En caso de aceptar cuando falta completar el saldo muestra el mensaje de saldo no suficiente y muestra cuánto dinero falta.
Se encarga de mostrar texto y en pantalla las cantidades o números.
Si está dispensando producto activa la “puertecilla” e inicia el timer para dispensar el producto.

```
1 referencia
private void btAceptar_Click(object sender, EventArgs e)
{
    contexto.Confirmar();

    // si aún falta dinero
    if (contexto.SState is StateRecibirDinero)
    {
        MostrarMensajeValor(contexto.mensaje, contexto.faltante);
        return;
    }

    MostrarMensaje(contexto.mensaje);

    //si esta dispensando
    if (contexto.SState is StateDispensando)
    {
        MostrarProducto(); //simula la entrega
        tmDispensar.Start();
    }
}
```

Botón cancelar

Se encarga de cancelar la operación en la que se encuentra, en caso de estar dispensando marca que no se permite, guarda en una variable si hay saldo a reembolsar y verifica si se debe realizar un reembolso y cierra el formulario de monedas y billetes.

```
//cancela la operación actual
1 referencia
private void btCancelar_Click(object sender, EventArgs e)
{
    //si está dispensando, no permitir cancelar
    if (contexto.SState is StateDispensando)
    {

        MostrarMensaje("\nOperación en proceso");

        return;
    }

    int reembolso = contexto.saldo;

    contexto.Cancelar();

    MostrarMensaje(contexto.mensaje);

    //si dinero antes de cancelar
    if (reembolso > 0)
    {
        MostrarCambio(reembolso);
    }

    //cerrar forms de monedas y billetes
    if (monedas != null && !monedas.IsDisposed)
        monedas.Close();

    if (billetes != null && !billetes.IsDisposed)
        billetes.Close();
}
```

Temporizador para dispensar

Simula el tiempo que tarda el producto en ser dispensado, ejecuta el estado actual, regresa el panel a su color inicial, actualiza el stock y si hay cambio para entregar.

```
//simula el tiempo que tarda en dispensarse el producto
1 referencia
private void TmDispensar_Tick(object sender, EventArgs e)
{
    tmDispensar.Stop();

    //ejecuta dispensando o recibiendo dinero
    contexto.Confirmar();

    LimpiarPanel();

    MostrarStock();

    //guarda temporal el cambio
    int cambioTemp = contexto.cambio;

    //muestra el estado del proceso
    MostrarMensaje(contexto.mensaje);

    //si hay cambio
    if (cambioTemp > 0)
    {
        //muestra charola
        MostrarCambio(cambioTemp);
    }

    //limpia el cambio
    contexto.cambio = 0;
}
```

Validación de etiquetas

Primero verifica si la máquina está disponible (en espera) y después verifica que el número ingresado sea de dos dígitos para consultar un producto correspondiente a la etiqueta.

```
//verifica que al consultar haya dos digitos
10 referencias
public void ValidarEtiqueta()
{
    //permite consultar cuando la maquina este en espera
    if (!(contexto.SState is StateEspera))
        return;

    //la consulta se realiza cuando se tiene dos digitos
    if (txtPantalla.Text.Length == 2)
    {
        string etiqueta = txtPantalla.Text;

        contexto.ConsultarPrecio(etiqueta);

        lblPantalla.Text = "\nConsultando precio...";

        tmLimpiar.Stop();
        tmPrecio.Start();
    }
}
```

Monedas

Al presionar la ranura de las monedas, se desplegará una segunda forma que contendrá 4 monedas con diferente denominación pudiendo elegir entre ellas para completar el saldo y adquirir un producto.

Código

Inicia haciendo referencia al formulario principal ya que necesita comunicarse con él para actualizar la pantalla. Y lo recibe para después guardar esa referencia.
Al presionar una moneda de la denominación que sea está enviará su valor al contexto y después actualizará el valor mostrado en pantalla.

```
//vista de las monedas
4 referencias
public partial class Form2 : Form
{
    //referencia a form1
    Form1 form1;

    //recibe como referencia al form1
    1 referencia
    public Form2(Form1 forma1)
    {
        InitializeComponent();

        //guarda el acceso al form1
        form1 = forma1;
    }

    //ingresa moneda de 1
    1 referencia
    private void pbMoneda1_Click(object sender, EventArgs e)
    {
        //envia el valor de la moneda al contexto
        form1.contexto.IngresarDinero(1);

        //actualiza el saldo en pantalla
        form1.MostrarSaldo();
    }
```

Billetes

Al presionar la ranura de los billetes, se desplegará una segunda forma que contendrá 3 billetes con diferente denominación pudiendo elegir entre ellos para completar el saldo y adquirir un producto.

Código

Inicia haciendo referencia al formulario principal ya que necesita comunicarse con él para actualizar la pantalla. Y lo recibe para después guardar esa referencia.
Al presionar un billete de la denominación que sea está enviará su valor al contexto y después actualizará el valor mostrado en pantalla.

```
//visualizacion de los billetes
4 referencias
public partial class Form3 : Form
{
    //referencia a form1
    Form1 form1;

    1 referencia
    public Form3(Form1 forma1)
    {
        InitializeComponent();

        //guarda el acceso al form1
        form1 = forma1;
    }

    //ingresa billete de 20
    1 referencia
    private void pbBillete20_Click(object sender, EventArgs e)
    {
        // envia el valor del billete al contexto
        form1.contexto.IngresarDinero(20);

        //actualiza el saldo en pantalla
        form1.MostrarSaldo();
    }
```

Implementación del patrón State

State

En interface se declaran todos los eventos o acciones posibles que se pueden presentar en el sistema y que sepa qué hacer si ocurre alguna de ellas.

```
using System;

namespace MaquinaExpEstado03
{
    //establece los métodos que debe llevar cada estado
    6 referencias
    public interface State
    {
        5 referencias
        void IngresarDinero(Context contexto, int cantidad);
        5 referencias
        void ConsultarPrecio(Context contexto, string posicion);
        5 referencias
        void ElegirProducto(Context contexto, string posicion);
        5 referencias
        void Confirmar(Context contexto);
        5 referencias
        void Cancelar(Context contexto);
    }
```

Context

Context se encarga de mantener el estado actual de los métodos pero no establece ningún comportamiento sino que delega los métodos al estado actual y que este se responsabilice del comportamiento de cada uno de ellos.

Constructor

Esta línea es muy importante ya que es el encargado de manejar la máquina, es el que sabe en qué estado se encuentra la máquina en cierto momento, pero no realiza ninguna acción sino que delega la responsabilidad al estado correspondiente para que ejecute las acciones o eventos necesarios.
Tiene como parámetro a la interfaz para obtener un estado válido cualquiera, se le asigna una variable y solo lo recibe al estado.
Y posteriormente guarda el estado actual.

```
28 referencias
public class Context //-------------------------------CONTEXT
{
    //constructor, estado inicial
    1 referencia
    public Context(State sstate)
    {
        SState = sstate;
    }

    //estado actual
    18 referencias
    public State SState { get; set; }
```

Métodos que pertenecen a context

Estos métodos permiten delegar la acción al estado actual del sistema, y cada uno de los estados va contener como deben comportarse esas acciones.

```
//métodos que delegan la acción al estado actual
7 referencias
public void IngresarDinero(int cantidad)
{
    SState.IngresarDinero(this, cantidad);
}

1 referencia
public void ConsultarPrecio(string posicion)
{
    SState.ConsultarPrecio(this, posicion);
}

0 referencias
public void ElegirProducto(string posicion)
{
    SState.ElegirProducto(this, posicion);
}

2 referencias
public void Confirmar()
{
    SState.Confirmar(this);
}

1 referencia
public void Cancelar()
{
    SState.Cancelar(this);
}
```

Reiniciar

Permite limpiar las variables que se necesitan para las operaciones y devuelve a un estado inicial.

```
//reiniciar
//limpia los valores temporales y regresa a un estado inicial
2 referencias
public void Reiniciar()
{
    saldo = 0;
    precio = 0;
    cambio = 0;
    faltante = 0;
    seleccion = "";

    SState = new StateEspera();
}
```

Estado Espera

Ingresar dinero

Cuando la máquina se encuentra en espera por una acción, verifica si se ha consultado un producto antes de ingresar dinero, actualiza el saldo actual del dinero cuando se ingresa una denominación y verifica si el saldo es igual o mayor al costo del producto o si el usuario debe seguir ingresando dinero.

En caso de completar el saldo total pasa al estado de producto listo, donde se confirma el producto a despachar o en su defecto sigue ingresando dinero y pasa al estado de recibiendo dinero.

```
5 referencias
public class StateEspera : State //----------------------------------ESPERA
{
    2 referencias
    public void IngresarDinero(Context contexto, int cantidad)
    {
        //verifica si se consulto un producto
        //si no se ha consultado no permite ingresar dinero
        if (contexto.precio == 0)
        {
            contexto.mensaje = "Primero consulte producto";
            return;
        }

        //asigna el dinero ingresado al saldo actual
        contexto.saldo += cantidad;

        //si el saldo es igual o mayor al costo
        if (contexto.saldo >= contexto.precio)
        {
            //muestra que ya se completó el saldo
            contexto.mensaje = "\nSaldo completo";
            //cambia al estado donde puede confirmar compra
            contexto.SState = new StateProductoListo();
        }
        else
        {
            //si aun falta completar el saldo
            contexto.mensaje = "\nRecibiendo dinero...";
            //continua recibiendo dinero
            contexto.SState = new StateRecibirDinero();
        }
    }
```

Consultar precio

Al consultar el precio el usuario ingresa el número de la etiqueta que acompaña al producto, y se verifica que sean dos dígitos para pasar a mostrar el costo o en caso de que ingrese el número de una etiqueta no válida se mostrará como producto no existente.
Como también se maneja el stock de los productos, en caso de que un producto esté agotado y se consulte se da el mensaje de que está agotado el producto.

```
2 referencias
public void ConsultarPrecio(Context contexto, string posicion)
{
    //guarda temporal la etiqueta del producto consultado
    contexto.seleccion = posicion;

    //obtiene el precio y stock del producto correspondiente a la etiqueta
    int precio = contexto.ObtenerPrecio(posicion);
    int cantidad = contexto.ObtenerStock(posicion);

    //para cuando no existe producto
    if (precio == 0)
    {
        contexto.precio = 0; //quita el precio anterior
        contexto.mensaje = "\nProducto no disponible";
        return;
    }

    //verifica el stock, en caso de producto agotado
    if (cantidad == 0)
    {
        contexto.precio = 0;
        contexto.mensaje = "\nProducto agotado";
        return;
    }

    contexto.precio = precio;
}

2 referencias
public void ElegirProducto(Context contexto, string posicion) { }
```

Confirmar

En caso de que se presione aceptar antes de ingresar dinero se le pedirá al usuario que realice la acción, o en caso de que presione aceptar sin haber realizado una acción previa también se mostrará.

```
//aceptar antes de ingresar dinero
2 referencias
public void Confirmar(Context contexto)
{
    contexto.mensaje = "\nIngrese dinero";
}

//cancelar sin haber realizado alguna operación
2 referencias
public void Cancelar(Context contexto)
{
    contexto.mensaje = "\nSin operación";
}
}
```

Estado Recibir dinero

Ingresar dinero

En este método se actualiza el saldo cada vez que el usuario ingresa dinero. Si el saldo es mayor o igual al costo del producto pasa al estado de confirmar producto o por el contrario se queda en el mismo estado.

```
2 referencias
public class StateRecibirDinero : State //------------------------RECIBIR DINERO
{
    2 referencias
    public void IngresarDinero(Context contexto, int cantidad)
    {
        //agrega el valor de monedas o billetes al saldo actual
        contexto.saldo += cantidad;

        //si el saldo es mayor o igual al costo
        if (contexto.saldo >= contexto.precio)
        {
            contexto.mensaje = "\nSaldo completo";
            //cambia al estado de confirmar compra
            contexto.SState = new StateProductoListo();
        }
        else
        {
            //si no se ha completado el costo
            //sigue en el mismo estado
            contexto.mensaje = "\nRecibiendo dinero...";
        }
    }
```

Confirmar

En caso de que se haya ingresado dinero, pero no haya cubierto el saldo completo si se presiona aceptar se mostrará el mensaje de que el saldo no es suficiente y el usuario deberá completar la cantidad. Se acompaña de la cantidad que falta.

```
2 referencias
public void Confirmar(Context contexto)
{
    //Si se confirma antes de completar el saldo
    contexto.faltante = contexto.precio - contexto.saldo;

    //alerta de que falta dinero
    contexto.mensaje = "Saldo insuficiente \nFalta:";
}
```

Cancelar

Si el usuario cancela la operación después de haber ingresado dinero pero sin haber completado el saldo total se le hará un reembolso del saldo acumulado, se limpian los valores y regresa al estado inicial.

```
2 referencias
public void Cancelar(Context contexto)
{
    //si hay dinero ingresado antes de completar el saldo
    if (contexto.saldo > 0)
    {
        contexto.mensaje = "\nReembolso Dinero";
    }

    //termina la compra
    //limpia y regresa al estado inicial
    contexto.Reiniciar();
}
}
```

Estado Producto Listo

Elegir producto

Se actualiza el número de etiqueta, la posición del producto.

```
2 referencias
public void ElegirProducto(Context contexto, string posicion)
{
    //actualiza la posición del producto
    contexto.seleccion = posicion;
}
```

Confirmar

Se confirma la compra del producto.

```
2 referencias
public void Confirmar(Context contexto)
{
    //calcula el cambio a devolver
    contexto.cambio = contexto.saldo - contexto.precio;

    contexto.mensaje = "\nEntregando producto...";

    //inicia a dispensar
    contexto.SState = new StateDispensando();
}
```

Cancelar

En caso de que el usuario quiera cancelar antes de confirmar se le hará un reembolso del saldo completo. Limpia los valores temporales y regresa al estado inicial.

```
2 referencias
public void Cancelar(Context contexto)
{
    //en caso de cancelar antes de aceptar la compra
    //pero despues de haber completado el saldo
    //reembolsa el saldo completo
    if (contexto.saldo > 0)
    {
        contexto.mensaje = "\nReembolso Saldo";
    }

    //termina la compra
    //limpia y regresa al estado inicial
    contexto.Reiniciar();
}
}
```

Estado Dispensando

Confirmar

Este método incluye la simulación de un producto atorado, ya sea que el resorte esté fallando o que el producto se atore entre otro producto o el vidrio, cualquiera que sea el caso el producto no se descuenta del stock.
Se agrega como random la probabilidad de que un producto se atore, muestra el mensaje de que se atoro el producto y devuelve el saldo completo.
En caso de que el producto se despache sin fallos, se descuenta del stock.
Se limpian los valores y regresa al estado inicial.
Y regresa al estado de espera.

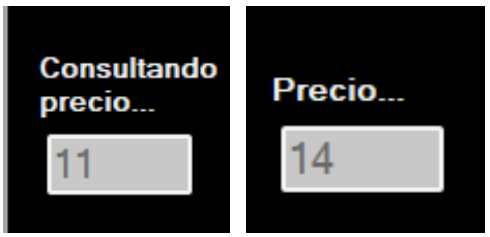
```
2 referencias
public void Confirmar(Context contexto)
{
    //random para simular la falla de que se atore el producto
    Random r = new Random();
    int falla = r.Next(1, 11);

    if (falla <= 5) //50% de que se atore un producto
    {
        contexto.mensaje = "\nProducto atorado";
        contexto.cambio = contexto.saldo; //devuelve todo el saldo
    }
    else //producto despachado sin inconvenientes
    {
        //se descuenta el producto del inventario
        switch (contexto.seleccion)
        {
            case "11": contexto.stock[0]--; break;
            case "15": contexto.stock[1]--; break;
            case "18": contexto.stock[2]--; break;
            case "19": contexto.stock[3]--; break;
            case "21": contexto.stock[4]--; break;
            case "23": contexto.stock[5]--; break;
            case "26": contexto.stock[6]--; break;
            case "28": contexto.stock[7]--; break;
            case "30": contexto.stock[8]--; break;
            case "33": contexto.stock[9]--; break;
            case "35": contexto.stock[10]--; break;
            case "37": contexto.stock[11]--; break;
            case "44": contexto.stock[12]--; break;
            case "45": contexto.stock[13]--; break;
            case "48": contexto.stock[14]--; break;
            case "49": contexto.stock[15]--; break;
        }
        contexto.mensaje = "\nProducto entregado";
    }

    //limpia y regresa al estado inicial
    contexto.saldo = 0;
    contexto.precio = 0;
    contexto.seleccion = "";

    //regresa al estado de espera
    contexto.SState = new StateEspera();
}
```

Pruebas

Caso 1

Consultar precio

El usuario ingresa el número que corresponde a una etiqueta que identifica la posición de un producto y devuelve el precio del mismo.



Caso 2

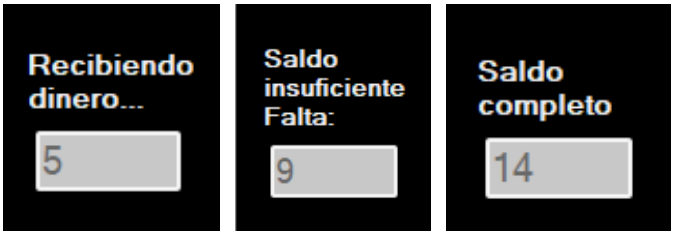
Consultar precio de un producto no disponible

Al consultar el precio de un producto e ingresar el número de una etiqueta que no existe dentro de la máquina lo marcará como producto no disponible.

Caso 3

Aceptar con saldo insuficiente

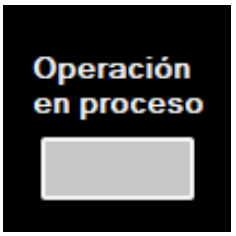
Al ingresar el dinero, si el usuario presiona aceptar para iniciar la compra y el dinero no es el monto completo, la máquina alerta que el saldo es insuficiente y cuál es la cantidad faltante.



Caso 4

Intentar cancelar mientras se está dispensando el producto

Después de aceptar continuar con la compra el producto inicia a despacharse por lo cual el usuario ya no puede cancelar la operación.



Caso 5

Cancelar mientras se tiene saldo incompleto

Después de consultar el precio de un producto e iniciar a ingresar dinero, si el usuario no está seguro de su compra puede interrumpir el proceso de ingresar el dinero y la máquina realiza la devolución del saldo acumulado en la inserción.

Caso 6

Cancelar con el saldo completo

Después de haber consultado el precio de un producto e ingresar el monto completo correspondiente al precio del producto y cancelar antes de aceptar la máquina realiza un reembolso del saldo completo.

Conclusión

El implementar el patrón state en este proyecto me resultó emocionante al inicio ya que es un muy buen tema de proyecto e ideal para el patrón, pero conforme avanzaba me di cuenta que puede resultar extenso y laborioso ya que se debe tomar en cuenta cada una de las posibles situaciones que se pueden presentar en dicho sistema.

Es por esto mismo que me parece un patrón muy importante, ya que una máquina o un sistema programable no sabe cómo comportarse recién es creado, sino que debe ser “alimentado” programado de manera detallada para que cumpla con las expectativas, contemple la seguridad de los usuarios y su óptimo funcionamiento.